

Module 14) Python – Collections, functions and Modules

6. Dictionaries

- **Introduction to dictionaries: key-value pairs**

What Are Key-Value Pairs?

A **key-value pair** is a set where:

- The **key** is a unique identifier.
- The **value** is the data or information associated with that key.

Example in Python:

```
person = {  
    "name": "Alice",  
    "age": 30,  
    "city": "New York"  
}
```

Here:

- "name" is a key, and "Alice" is its value.
- "age" is a key, and 30 is its value.
- "city" is a key, and "New York" is its value.

Key Characteristics

1. **Uniqueness of Keys:**

Keys must be unique within a dictionary. Duplicate keys are not allowed.

2. **Key Types:**

Keys must be of an immutable type (e.g., strings, numbers, tuples).

Mutable types like lists are not allowed as keys.

3. **Value Types:**

Values can be of any type—numbers, strings, lists, other dictionaries, etc.

4. **Unordered** (prior to Python 3.7):

In older versions, dictionaries did not maintain order. From Python 3.7+, the insertion order is preserved.

- **Accessing, adding, updating, and deleting dictionary elements**

Accessing Dictionary Elements

You can access the value associated with a key using **square brackets []** or the **.get() method**

```
person = {"name": "Alice", "age": 30}
print(person["name"]) # Output: Alice
```

Adding Elements

You can add a new key-value pair simply by assigning a value to a new key.

```
person["email"] = "alice@example.com"
```

Updating Elements

If the key **already exists**, assigning a new value to it will **update** the old value.

```
person["age"] = 31 # Updates age from 30 to 31
```

Deleting Elements

There are several ways to delete elements from a dictionary:

```
del person["email"] # Deletes the key 'email'
```

- **Dictionary methods like `keys()`, `values()`, and `items()`.**

`keys()` Method

The `keys()` method returns a **view object** that contains all the keys in the dictionary.

```
person = {"name": "Alice", "age": 30, "city": "New York"}
```

```
print(person.keys())
```

```
# Output: dict_keys(['name', 'age', 'city'])
```

`values()` Method

The `values()` method returns a view object containing all the **values** in the dictionary.

```
print(person.values())
```

```
# Output: dict_values(['Alice', 30, 'New York'])
```

items() Method

The items() method returns a view object with all **key-value pairs** as **tuples**.

```
print(person.items())
```

```
# Output: dict_items([('name', 'Alice'), ('age', 30), ('city', 'New York')])
```